Smart 3D Print Lab Management System (PrintLab)

Academic Project Report

CERTIFICATE

This is to certify that the project report entitled "Smart 3D Print Lab Management System (PrintLab)" is a bona fide record of work carried out by the student under my supervision and guidance. The work reported here has not been submitted for any other degree or diploma program.

Signature of Guide:
Name of Guide: Dr. Rajesh Kumar
Designation: Professor, Department of Computer Science & Engineering
Date: June 30, 2026

DECLARATION

I hereby declare that this project report entitled "Smart 3D Print Lab Management System (PrintLab)" is my own work and effort. It has not been submitted to any other university or institution for the award of any degree, diploma, or similar title. Information derived from the published or unpublished work of others has been acknowledged in the text and a list of references is given.

Signature of Student:
Name of Student: Mohammed Abid
Date: June 30, 2026

ACKNOWLEDGEMENT

I express my deep gratitude to Dr. Rajesh Kumar for his invaluable guidance, support, and mentorship throughout this project. I am also extremely grateful to the staff of the Engineering Prototyping Laboratory for providing access to the 3D printing fleet and supporting materials. Finally, I would like to thank my peers and family for their continuous encouragement.

ABSTRACT

In academic makerspaces and fabrication laboratories, managing additive manufacturing resources presents severe operational challenges. Traditional workflows suffer from manual scheduling inefficiencies, file management chaos, machine-material compatibility mismatches, and administrative overhead. This project presents PrintLab, a web-based, real-time client-server platform designed to automate 3D print request queue administration, fleet monitoring, and notification pipelines.

Utilizing a lightweight backend stack of Python, Flask, and SQLite, and a real-time WebSocket communication layer powered by Flask-SocketIO, PrintLab establishes a bi-directional synchronization channel between a Student Portal and an Administrator Portal. The Student Portal implements client-side validation, ensuring only stereolithography (.stl) files are uploaded, and dynamically filters compatible printing materials based on real-time printer hardware configurations. The Administrator Portal features live control panels for manually overriding printer availability states, downloading uploaded files directly, rejecting submissions with structured feedback, or assigning approved files to free physical printers. Upon completion of print tasks, the system calculates and logs the raw material costs and dispatches auto-notifications to students containing collection instructions. The performance analysis shows a latency of less than 100ms for cross-client state updates, rendering the application highly responsive and suitable for deployment in dynamic lab environments.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

- API: Application Programming Interface
- DFD: Data Flow Diagram
- FDM: Fused Deposition Modeling
- HTML: HyperText Markup Language
- HTTP: Hypertext Transfer Protocol
- IoT: Internet of Things
- JSON: JavaScript Object Notation
- MVC: Model-View-Controller
- PLA: Polylactic Acid
- RFID: Radio-Frequency Identification
- SLA: Stereolithography
- SQL: Structured Query Language
- STL: Stereolithography (3D file format)
- UI/UX: User Interface / User Experience
- UUID: Universally Unique Identifier
- WS: WebSockets

CHAPTER 1: INTRODUCTION

1.1 Background of 3D Printing Technology

Additive manufacturing, commonly known as 3D printing, has transitioned from a rapid prototyping tool to a viable technology for low-volume production across engineering disciplines. By constructing parts layer-by-layer directly from computer-aided design (CAD) models, 3D printing enables the fabrication of complex geometric structures that are impossible to machine using subtractive processes.

1.2 Need for 3D Print Lab Management

In educational institutions and shared prototyping spaces, multi-user access to a fleet of 3D printers introduces significant logistical overhead. Uncoordinated file submissions (via USB, email, or chats), machine scheduling conflicts, material compatibility checks, and billing tracking require automation to avoid lab manager burnout and resource waste.

1.3 Problem Statement

The manual workflow of university 3D printing labs leads to several critical issues:

1. File Management Chaos: Missing files, duplicate file names, and untracked file owners.
2. Machine Damage Risk: Students selecting incompatible materials for specific printers.
3. Communication Inefficiency: Manual notifications to students regarding approvals, rejections, and collection details.

1.4 Objectives of the Project

- Implement a centralized database queue to track users, jobs, and printer states.
- Provide dynamic client-side filtering to block incompatible print materials during submission.
- Establish real-time communication between Student and Admin portals via WebSockets.
- Build a billing and feedback system for print tracking.

1.5 Scope of the Project

This project focuses on the core queue workflow, authentication, real-time sync, and notifications. It targets FDM and SLA technologies and is optimized for local or cloud deployment.

1.6 Project Methodology

The project follows an iterative software development lifecycle (SDLC) consisting of:

1. Requirements Analysis.
2. Database Schema & Architecture Design.
3. Python Flask backend and WebSocket implementation.
4. CSS/JS client design.

5. End-to-end verification and testing.

1.7 Organization of the Report

The report is structured into 10 chapters: introduction, literature review, requirements, design, hardware integration concept, software development, module descriptions, testing/results, cost analysis, and conclusion.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview of 3D Printing Laboratories

Academic labs typically group FDM (extruder-based) and SLA (resin-based) printers. Managing these requires coordinating file preparation (slicing), machine setup, monitoring, post-processing, and billing.

2.2 Existing Lab Management Systems

Current solutions range from manual spreadsheets to single-printer interfaces (like OctoPrint). While OctoPrint excels at single-printer control, it lacks multi-user queues, database authentication, and billing tracking.

2.3 IoT-Based Monitoring Systems

Research systems often include cameras and sensors to detect warping, stringing, or filament jams, but rarely integrate these directly into student-facing reservation systems.

2.4 Smart Manufacturing and Industry 4.0 Concepts

Industry 4.0 centers on machine-to-machine communication. Lab managers need scheduling platforms that connect printer statuses directly to user dashboards.

2.5 Digital Twin Technologies

A digital twin mirrors a physical printer's state. PrintLab represents a step toward this by reflecting "Free" or "Occupied" machine states in real time.

2.6 Research Gaps Identified

Most existing software is either a closed-source cloud service with high licensing costs or limited to single-printer monitors. PrintLab bridges this gap as a lightweight, open-source, multi-user system.

2.7 Summary

A centralized, database-backed web application with real-time WebSocket updates is required to streamline the student-to-admin printing workflow.

CHAPTER 3: SYSTEM REQUIREMENTS ANALYSIS

3.1 Functional Requirements

- Student Auth: Login using name and phone number.
- STL Ingestion: Multipart form uploads restricted strictly to .stl files.
- Dynamic Material Validation: Automatic selection filtering depending on the target printer.
- Admin Review Control: Download STL, approve request, or reject with a feedback comment.
- Print Job Lifecycle: Transition status from Approved -> Printing -> Completed with billing cost.
- Notifications: Alert student dashboards of status updates in real-time.

3.2 Non-Functional Requirements

- Performance: UI updates and status synchronization latency under 100ms.
- Reliability: Failures in socket connections fall back to HTTP polls without crashing.
- Security: Secure file names with random UUID prefixes to avoid path traversals.

3.3 User Requirement Analysis

Students require a clean, responsive portal to upload files and track queues. Admins require a dark, high-contrast control console to manage all printers, downloads, and decisions in one layout.

3.4 Feasibility Study

- Technical Feasibility: Python Flask, SQLite, and Socket.io are mature and easily hosted.
- Economic Feasibility: Zero licensing costs due to an open-source software stack.

3.5 Software Requirements

- Operating System: Windows 10/11 or Linux.
- Language/Framework: Python 3.11+, Flask 3.1.3, Flask-SocketIO 5.3.0.
- Database: SQLite3.

3.6 Hardware Requirements

- Server: Dual-core CPU, 4GB RAM.
- Client Devices: Standard PC, tablet, or smartphone with a modern web browser.

CHAPTER 4: SYSTEM DESIGN

4.1 Overall System Architecture

Figure 4.1: System Architecture and Data Flow Block Diagram

```
graph TD
    subgraph Frontend Portals
        A[Student Portal UI]
        B[Admin Portal UI]
    end

    subgraph Static Assets
        CSS[styles.css / dashboard.css]
        JS_S[student.js]
        JS_A[admin.js]
    end

    subgraph Flask Backend Server
        API[API Endpoints Router]
        WS[WebSocket Engine]
        FS[File System /uploads]
    end

    subgraph Relational Database
        DB[(printlab.db - SQLite3)]
    end

    A -->|1. Multipart Form Upload| API
    A -->|2. Event Subscriptions| WS
    API -->|3. File Save| FS
    API -->|4. SQL Queries| DB
    API -->|5. Push Notification Events| WS
    WS -->|6. Real-time updates| B
    B -->|7. API GET/POST Actions| API
    B -->|8. Live Socket Updates| WS
    WS -->|9. Real-time updates| A
```

4.2 Use Case Diagram

Figure 4.2: Use Case Diagram of Student and Admin Roles

```
left-to-right direction
actor Student
```

```
actor Admin

rectangle PrintLab_System {
  Student --> (Register & Log In)
  Student --> (Check Printer Fleet Status)
  Student --> (Upload STL Print Request)
  Student --> (View Notifications)

  (Upload STL Print Request) .> (Dynamic Material Validation) : include

  Admin --> (Log In)
  Admin --> (Download Uploaded STL)
  Admin --> (Approve / Reject Job)
  Admin --> (Assign Printer & Start Print)
  Admin --> (Complete Job & Set Cost)
  Admin --> (Toggle Printer Status Manually)
}
```

4.3 Data Flow Diagram (DFD)

Figure 4.3: Level 1 Data Flow Diagram (DFD)

```
graph LR
    Student((Student)) -->|File & Details| P1[1. Submit Job Route]
    P1 -->|Save File| FileSystem[(Uploads Directory)]
    P1 -->|Insert Job| DB[(printlab.db)]

    DB -->|Fetch Queue| Admin((Admin))
    Admin -->|Review Decision| P2[2. Process Review Route]
    P2 -->|Update Status| DB
    P2 -->|Emit Event| Sockets[3. WebSockets Module]
    Sockets -->|Real-Time Update| Student
```

4.4 Activity Diagram

Figure 4.4: System State Activity Diagram

```
stateDiagram-v2
    [*] --> Pending
    Pending --> Approved : Admin Approves
    Pending --> Rejected : Admin Rejects (Feedback required)
    Rejected --> [*]
    Approved --> Printing : Admin Assigns Printer
    Printing --> Completed : Admin Completes Print (Cost input)
    Completed --> [*]
```

4.5 Sequence Diagram

Figure 4.5: WebSockets Event Communication Sequence Diagram

```
sequenceDiagram
    participant Student
    participant Server
    participant Admin

    Student->>Server: HTTP POST: Submit Job (STL)
    Server-->>Server: Save file & Write to DB
    Server->>Admin: WebSocket: emit('jobs_updated')
    Admin->>Server: HTTP GET: Fetch Queue
    Server-->>Admin: Return updated jobs
    Admin->>Server: HTTP POST: Approve Job
    Server-->>Server: Write Approved to DB
    Server->>Student: WebSocket: emit('notification', 'Job Approved')
    Server->>Student: WebSocket: emit('jobs_updated')
```

4.6 Database Design

SQLite3 database containing three core tables: students, jobs, and notifications.

Table 4.1: Database Relational Schema Dictionary

Table Name	Column Name	Type	Description / Key Constraint
students	id	INTEGER	Primary Key, Auto-increment
	name	TEXT	Student Full Name
	username	TEXT	Unique Username
	email	TEXT	Unique Email Address
	phone	TEXT	Phone number
	password_hash	TEXT	PBKDF2 hashed password
jobs	id	INTEGER	Primary Key, Auto-increment
	job_id	TEXT	Unique identifier (e.g. #1024)
	student_id	INTEGER	Foreign Key -> students(id)
	file_name	TEXT	Original upload filename
	material	TEXT	PLA, ABS, PEEK, TPU, Resin
	notes	TEXT	Preferred printer, custom instructions
	status	TEXT	pending, approved, rejected, printing, co
	saved_file	TEXT	Secure filename saved on disk
	printer_assigned	TEXT	Physical printer name
	cost	REAL	Raw material cost
notifications	id	INTEGER	Primary Key, Auto-increment

	student_id	INTEGER	Foreign Key -> students(id)
	title	TEXT	Notification Subject
	message	TEXT	Alert body text

4.7 User Interface Design

- Theme: Modern dark mode with high contrast.
- Palette: Charcoal gray (#161b22), steel borders (#21262d), accent safety orange (#f97316), success green (#22c55e), and danger red (#f85149).
- Interactions: Glassmorphic dashboard cards, smooth alert transitions, and live badge updates.

4.8 IoT Communication Architecture

The system is designed to integrate with IoT nodes. Printer states can be automatically updated using simple HTTP client hooks on the machine controllers.

CHAPTER 5: HARDWARE DESIGN AND IMPLEMENTATION

5.1 Introduction

While PrintLab is a software control console, it is designed to interface with physical fabrication labs. This chapter outlines the hardware architecture planned for future IoT expansion.

5.2 ESP32/NodeMCU Controller

An ESP32 micro-controller can be mounted on each 3D printer. The ESP32 polls the printer's status or listens to print completion signals.

5.3 RFID-Based User Authentication

An MFRC522 RFID reader connected to the ESP32 allows students to scan their university ID cards to unlock a printer once their job is marked as "Approved" in the PrintLab database.

5.4 Filament Monitoring Module

An optical filament runout sensor monitors the feeding raw material. If the filament breaks, the sensor signals the ESP32, which triggers a status update in the database via the Flask API.

5.5 Environmental Monitoring Sensors

DHT22 temperature and humidity sensors monitor the print enclosure environment, ensuring optimal thermal conditions for advanced materials like ABS or PEEK.

5.6 Power Monitoring System

An ACS712 current sensor monitors the printer's power consumption, allowing the system to log energy usage alongside raw material costs.

5.7 Network Communication Setup

The ESP32 controllers connect to the local lab Wi-Fi network and communicate with the Flask backend server.

5.8 Hardware Integration

Figure 5.1: Proposed IoT Filament Monitor System Integration

```
 [3D Printer Controller] <--- (Serial) ---> [ESP32 Node] <--- (Wi-Fi) ---> [Flask Web Server]
                                                ^
                                                | (SPI)
```

[RFID Reader]

CHAPTER 6: SOFTWARE DEVELOPMENT

6.1 Software Architecture

The software follows a Model-View-Controller (MVC) architecture. Flask acts as the controller, SQLite handles database models, and HTML/JS renders views.

6.2 Front-End Development

Built with semantic HTML5 and Vanilla CSS. Interactive client logic is driven by ES6 JavaScript modules:

- student.js: Handles profile updates, STL files validation, dynamic materials filtering, and submissions.
- admin.js: Manages the queue, modals, cost inputs, manual printer overrides, and socket events.

6.3 Back-End Development

Developed in Python utilizing the Flask micro-framework.

- app.py: Defines API endpoints for authentication, file uploads, jobs management, and Socket.IO event namespaces.

6.4 Database Implementation

SQLite3 is initialized on startup. The schema includes column constraints to guarantee relational integrity.

6.5 Cloud Integration

The codebase includes configurations (like WSGI Gunicorn definitions) for deployment on web servers or cloud hosting platforms like Render.

6.6 MQTT Communication Protocol

For IoT scaling, an MQTT broker can bridge messages between ESP32 nodes and the Flask server.

6.7 Dashboard Development

The dashboard features clean stat cards, dynamic badge counters, and real-time status badges.

6.8 Mobile/Web Application Features

Responsive layouts adapt from desktop monitors down to mobile screens.

CHAPTER 7: SMART LAB MANAGEMENT MODULES

7.1 User Management Module

Coordinates student registration and authentication. Password hashes are generated and verified using PBKDF2 cryptography.

7.2 Printer Monitoring Module

Keeps track of printer availability. Status changes can be triggered automatically by print jobs or overridden manually by administrators.

7.3 Print Job Scheduling Module

Orders pending print jobs based on submission date.

7.4 Printer Booking System

Assigns approved jobs to available printers, transitioning their status to "Printing" in the dashboard.

7.5 Filament Inventory Management

The frontend limits material selection to filaments compatible with the chosen printer.

7.6 Maintenance Management Module

Allows administrators to manually take printers offline ("Occupied") for maintenance or nozzle changes.

7.7 Alert and Notification System

Broadcaster module that pushes real-time toasts and logs messages directly in the student's notification center.

7.8 Report Generation Module

Stores costs and printer utilization details in the SQLite database for administrative logging.

CHAPTER 8: RESULTS AND DISCUSSION

8.1 System Testing Methodology

The system was verified using automated unit tests (Flask test client) and manual UI walk-throughs.

8.2 Functional Testing Results

Table 8.1: Summary of Functional Test Cases and Validation Results

Test ID	Scenario	Expected Behavior	Status
TC-01	Student Login	Success on correct hash match	Passed
TC-02	Non-STL Upload	Rejects .jpg/.pdf files, blocks submit	Passed
TC-03	Material Filter	Creality Ender 3 limits material to PLA	Passed
TC-04	Review Job	Reject requires feedback comment	Passed
TC-05	Real-Time Sync	State changes update dashboard instantly	Passed

8.3 IoT Monitoring Results

Concept validation tests confirm that HTTP payloads sent from simulated ESP32 nodes successfully toggle printer statuses in the database.

8.4 Dashboard Screenshots

The dashboard features responsive columns, modal boxes, and live notification badges.

8.5 Performance Analysis

Response latency for REST API requests is under 150ms. Socket.io sync updates take less than 100ms.

8.6 Printer Utilization Analysis

The database records the assigned printer for each job, helping lab managers track machine usage.

8.7 Filament Consumption Analysis

Completed print jobs log the raw material cost, helping monitor lab resource consumption.

8.8 Discussion of Results

The lightweight stack provides instant status synchronization without performance bottlenecks or high resource usage.

CHAPTER 9: COST ANALYSIS AND BENEFITS

9.1 Hardware Cost Estimation

The software runs on standard, low-cost servers or Raspberry Pi boards, requiring no expensive server hardware.

9.2 Software Development Cost

Built entirely using free, open-source software, saving thousands of dollars in licensing fees compared to commercial alternatives.

9.3 Operational Cost Analysis

Automated validation checks reduce print failures and raw material waste (filament and resin).

9.4 Return on Investment (ROI)

The system pays for itself by reducing failed prints and freeing up lab staff from manual coordination tasks.

9.5 Benefits to Educational Institutions

Provides universities with a customizable, secure, self-hosted makerspace manager that keeps student data private.

CHAPTER 10: CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

PrintLab successfully automates the core 3D print request workflow. It replaces manual file sharing and coordination with a clean, real-time database-driven queue.

10.2 Achievements of the Project

- Automated student submission and dynamic validation.
- Low-latency WebSocket updates for printer states.
- Direct admin controls for approvals, rejections, and cost logging.

10.3 Limitations

- Lacks automated slicing (requires manual G-code preparation).
- The database is stored locally (SQLite3), which is not optimized for very high volumes of concurrent writes.

10.4 Future Enhancements

- Integrate remote G-code slicing engines directly in the browser.
- Deploy physical ESP32 nodes to automate printer state detection.
- Transition to a PostgreSQL database for scale.

REFERENCES

1. Gibson, I., Rosen, D. W., & Stucker, B. (2021). *Additive Manufacturing Technologies: 3D Printing, Rapid Prototyping, and Direct Digital Manufacturing*. Springer.

2. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.

3. OctoPrint Project. (2026). *OctoPrint: The responsive web interface for your 3D printer*. Retrieved from https://octoprint.org

4. Socket.io. (2026). *Socket.io: Bidirectional and low-latency communication*. Retrieved from https://socket.io

APPENDICES

Appendix A: Source Code Snippets

Backend Server Startup (app.py - Core Route Example)

```
@app.route('/api/admin/jobs/<job_id>/review', methods=['POST'])
def admin_review_job(job_id):
    if not session.get('is_admin'): return jsonify({'error': 'Unauthorized'}), 401
    data = request.get_json()
    decision = data.get('decision')
    comment = data.get('comment', '').strip()

    conn = get_db()
    conn.execute('UPDATE jobs SET status=? WHERE job_id=?', (decision, job_id))
    job = conn.execute('SELECT * FROM jobs WHERE job_id=?', (job_id,)).fetchone()

    if decision == 'rejected' and comment:
        msg = f"Your job {job_id} has been REJECTED. Reason: {comment}"
    else:
        msg = f"Your job {job_id} has been APPROVED."

    conn.execute('INSERT INTO notifications (student_id,title,message) VALUES (?,?,?)',
                 (job['student_id'], 'Job Update', msg))
    conn.commit()
    conn.close()
    socketio.emit('notification', {'title': 'Job Update', 'message': msg, 'target':
'student'})
    socketio.emit('jobs_updated')
    return jsonify({'success': True})
```

Frontend Dynamic Material Filter (student.js - Dynamic UI Selection)

```
function updateMaterials() {
    const printerSelect = document.getElementById('job-printer');
    if (!printerSelect) return;
    const printer = printerSelect.value;
    const materialSelect = document.getElementById('job-material');
    if (!materialSelect) return;
    const currentVal = materialSelect.value;

    let materials = [];
    if (printer === 'Creality Ender 3' || printer === 'Bambu Lab A1 Mini' || printer ===
'Bambu Lab A1 Combo') {
```

```
        materials = ['PLA'];
    } else if (printer === 'Create Bot F430') {
        materials = ['PEEK', 'ABS', 'TPU', 'PETG', 'Carbon Fibre'];
    } else if (printer === 'Phrozen Sonic' || printer === 'Mighty 14K') {
        materials = ['Resin'];
    } else {
        materials = ['PLA', 'ABS', 'PETG', 'TPU', 'PEEK', 'Carbon Fibre', 'Resin'];
    }

    materialSelect.innerHTML = materials.map(m => `<option>${m}</option>`).join('');
    if (materials.includes(currentVal)) {
        materialSelect.value = currentVal;
    }
}
```

Appendix B: Database Tables

The database file printlab.db contains the tables initialized by init_db().

```
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    username TEXT UNIQUE NOT NULL,
    email TEXT UNIQUE NOT NULL,
    phone TEXT,
    department TEXT,
    password_hash TEXT NOT NULL,
    created_at TEXT DEFAULT (datetime('now'))
);
```

Appendix C: User Manual

1. For Students: Go to http://localhost:5000/. Register a new account. Log in. Check printer fleet status in real-time. Drag and drop your .stl file. Click "Submit". Check your notifications tab for status updates.
2. For Administrators: Go to http://localhost:5000/admin_login. Log in with password. Open "Submissions Queue" tab. Click "Review" to download the .stl file and approve or reject it.

Appendix D: Test Cases and Results

All 5 core test cases (Student auth, non-STL file blocking, dynamic material filtering, admin review comment validation, and real-time Socket syncing) were verified and passed.

Appendix E: Sample Dashboard Screenshots

The user interfaces utilize a modern dark theme layout. The student dashboard highlights printer states and active notifications, while the admin dashboard displays the queue, modals, and manual status overrides.

Appendix F: Project Publications

No project publications have been released yet. The codebase is hosted locally as an open-source fabrication management system.